

Ship Routing and Scheduling in Industrial and Tramp Shipping

Chapter 13 — *Vehicle Routing: Problems, Methods, and Applications*, 2nd Edition (2014)

Marielle Christiansen Kjetil Fagerholt

Lecture Notes — Self-Contained Reading Version

2014

Chapter Overview I

This chapter covers the rich family of ship routing and scheduling problems that arise in **industrial** and **tramp** maritime transportation. Unlike road vehicle routing, ship routing involves enormous cargo quantities, multi-week planning horizons, complex contractual arrangements, and variable fuel costs that depend on speed in a strongly non-linear way.

Topics covered in this chapter:

- ➊ **Introduction** — the economic and environmental context of maritime shipping; the three main shipping segments.
- ➋ **Cargo Routing and Scheduling** — the core optimisation problem for industrial and tramp shipping; exact and heuristic solution methods.
- ➌ **Maritime Inventory Routing (MIR)** — integrating inventory management at ports with ship scheduling decisions.
- ➍ **Dynamic and Stochastic Ship Routing** — handling weather uncertainty, port congestion, and real-time cargo spot offers.
- ➎ **Conclusions and Future Directions.**

Key Takeaway of the Chapter

Maritime transportation is responsible for more than 80% of world trade by volume. Optimising ship routing and scheduling can reduce operational costs by 10–20% and substantially cut CO₂ emissions, making this one of the most impactful application areas of vehicle routing research.

Chapter Overview III

chapter_roadmap.png

Introduction: The Maritime Transportation Sector I

International seaborne trade handles more than **8 billion tonnes** of goods annually (UNCTAD 2011), making maritime transport the backbone of global commerce. The ship routing and scheduling problem is the operational challenge of deciding *which ship goes where, in what order, carrying what cargo, and at what speed*.

According to the book, international trade depends on maritime transportation for more than eight billion tonnes of goods per year. Shipping produces more CO₂ than aviation — motivating better routing for environmental as well as economic reasons.

Introduction: The Maritime Transportation Sector II

Three Shipping Segments — Definitions

Maritime transport is traditionally divided into three segments. Understanding which segment applies determines the structure of the optimisation problem.

Industrial shipping means the shipowner controls both the fleet *and* the cargo. A mining company, for instance, might own both the iron-ore mine and the ships that deliver the ore to steel plants. Because the company is optimising its *own* supply chain, the objective is to **minimise total transport cost**, and all cargoes must be served.

Liner shipping means ships operate on *fixed, published schedules* that they follow regardless of whether the ship is full — analogous to a scheduled airline or bus service. Ships visit the same sequence of ports (the “rotation”) every week or two weeks. Shippers pay per container slot. The optimisation problem is primarily one of **network design**: which ports to connect, at what frequency, with how many ships.

Tramp shipping means the ship follows the cargo, acting like a maritime taxi service. The tramp operator bids for cargoes on the spot market and holds long-term contracts, then decides which cargoes to accept and how to route the fleet to **maximise profit**. Accepted cargoes generate revenue; mandatory contract cargoes must be served (possibly by a hired spot ship if own fleet is insufficient).

Introduction: The Maritime Transportation Sector III

shipping_sectors.png

Section 13.2: Cargo Routing and Scheduling — Overview I

This section covers the core problem of **routing a heterogeneous fleet of ships to serve a set of cargoes**. The problem arises in both industrial and tramp shipping contexts and is fundamentally a *pickup-and-delivery* vehicle routing problem on a maritime network.

Unlike classical road VRP, the maritime pickup-and-delivery problem has several distinctive features that shape the mathematical formulation:

- 1 Cargo quantities are very large (thousands of tonnes), and a single ship may carry only *one* cargo at a time (full ship-load) or several smaller cargoes simultaneously.
- 2 Some cargoes are **mandatory** (long-term contracts that *must* be served, possibly by hiring a spot ship at extra cost). Others are **optional** (spot market cargoes that can be declined).
- 3 The planning horizon is typically *weeks to months*, not hours.
- 4 Each ship has its own *origin* (current position) and an *artificial destination* node representing the end of its planning period.
- 5 Time windows are specified at each port for both loading and discharging, often with *hard* limits enforced by tidal and terminal constraints.

Section 13.2: Cargo Routing and Scheduling — Overview II

pickup_delivery_network.png

The Maritime Pickup-and-Delivery Problem: Network Structure I

Problem Definition

We are given a fleet K of ships and a set N^C of *mandatory cargoes* together with a set N^O of *optional* (spot-market) cargoes. Each cargo i has a pickup port and a delivery port, a quantity Q_i (tonnes), and a revenue R_i per unit transported. The goal is to assign cargoes to ships and determine each ship's route and schedule to **maximise profit** (revenue minus voyage costs).

Network structure. For each ship $k \in K$ we build a directed graph. Let N_k^P denote the set of *pickup* nodes for ship k (one per cargo that ship k could potentially carry), and let N_k^D be the corresponding *delivery* nodes. In addition, each ship k has an artificial *origin* node $o(k)$ (its current location at the start of the planning period) and an artificial *destination* node $d(k)$ (representing “done”).

The Maritime Pickup-and-Delivery Problem: Network Structure II

Notation (read carefully)

- K — the set of all ships in the fleet.
- N^C — set of **mandatory** cargo indices (contracts that must be served).
- N^O — set of **optional** (spot market) cargo indices.
- $N_k = N_k^P \cup N_k^D \cup \{o(k), d(k)\}$ — all nodes relevant to ship k .
- A_k — set of **arcs** for ship k ; arc $(i, j) \in A_k$ means ship k can sail directly from node i to node j .
- Q_i — the **quantity** (load in tonnes) of cargo i .
- Q_k^C — the **capacity** (maximum load in tonnes) of ship k .
- R_i — the **revenue** earned per tonne of cargo i transported.
- C_{ijk} — the **voyage cost** for ship k on arc (i, j) (fuel, port fees, canal tolls, etc.).
- C_i^S — the **spot market cost** — the fee paid to hire an outside (spot) ship to carry mandatory cargo i if none of our own ships can serve it.

The Maritime Pickup-and-Delivery Problem: Network Structure III

Decision Variables

- $x_{ijk} \in \{0, 1\}$ — equals 1 if ship k sails *directly* from node i to node j ; 0 otherwise. The subscript ijk is read “x sub i-j-k.”
- $z_i \in \{0, 1\}$ — equals 1 if mandatory cargo $i \in N^C$ is served by a *spot market* ship rather than our own fleet.
- $y_i \in \{0, 1\}$ — equals 1 if optional cargo $i \in N^O$ is actually transported, 0 if declined.
- $t_{ik} \geq 0$ — the **time** at which ship k begins service at node i .
- $l_{ik} \geq 0$ — the **load** (weight in tonnes) on ship k when it *departs* from node i .

The MPDP Mathematical Model: Objective and Route Constraints I

The tramp ship routing and scheduling problem is formulated as a **Mixed-Integer Programme (MIP)**. We state the objective function and the key constraint groups, explaining each in plain language.

Objective Function (13.1) — Maximise Profit

$$\max \underbrace{\sum_{i \in N^C} R_i Q_i}_{\text{mandatory revenue}} + \underbrace{\sum_{i \in N^O} R_i Q_i y_i}_{\text{optional revenue}} - \underbrace{\sum_{k \in K} \sum_{(i,j) \in A_k} C_{ijk} x_{ijk}}_{\text{voyage costs}} - \underbrace{\sum_{i \in N^C} C_i^S z_i}_{\text{spot market cost}} \quad (13.1)$$

Here R_i is revenue per tonne, Q_i is quantity in tonnes, y_i (y sub i) equals 1 if optional cargo i is accepted, C_{ijk} is the voyage cost for ship k on leg (i, j) , x_{ijk} equals 1 if that leg is used, C_i^S is the spot-market surcharge for covering mandatory cargo i with a hired ship, and z_i equals 1 if that option is exercised.

The MPDP Mathematical Model: Objective and Route Constraints II

How to Read the Objective Function

The formula computes net profit: the first two terms are gross revenue (mandatory cargoes always earn revenue since they must be transported); the last two terms are costs. Hiring a spot ship ($C_i^S z_i$) is expensive, so the solver prefers to assign mandatory cargoes to own ships. However, if a mandatory cargo is geographically inconvenient, paying the spot premium may still be optimal. Optional cargoes (y_i) are accepted only if the extra revenue exceeds the extra voyage cost.

Constraint (13.2) — Every mandatory cargo is served:

$$\sum_{k \in K} \sum_{j \in N_k} x_{ijk} + z_i = 1 \quad \forall i \in N^C \quad (13.2)$$

The double sum counts how many of our ships pick up cargo i . Adding z_i (equals 1 if a spot ship covers it) gives exactly 1, ensuring the cargo is served once and only once.

The MPDP Mathematical Model: Objective and Route Constraints III

Constraint (13.3) — Optional cargo is accepted iff visited:

$$\sum_{k \in K} \sum_{j \in N_k} x_{ijk} - y_i = 0 \quad \forall i \in N^O \quad (13.3)$$

For optional cargo i : the indicator $y_i = 1$ if and only if some ship actually visits the pickup node. If no ship goes there, $y_i = 0$ and the cargo is simply not carried — there is no penalty.

Constraint (13.4) — Each ship departs its origin exactly once:

$$\sum_{j \in N_k} x_{o(k)jk} = 1 \quad \forall k \in K \quad (13.4)$$

This guarantees every ship actually departs from its starting position (it cannot “stay at home”).

Constraint (13.5) — Flow conservation (no teleportation):

$$\sum_{j \in N_k} x_{ijk} - \sum_{j \in N_k} x_{jik} = 0 \quad \forall k \in K, i \in N_k \setminus \{o(k), d(k)\} \quad (13.5)$$

The MPDP Mathematical Model: Objective and Route Constraints IV

For every intermediate node i (not the origin or the artificial destination), the number of times ship k *arrives* at i equals the number of times it *departs*. This prevents ships from “disappearing” or being duplicated.

The MPDP Mathematical Model: Load and Time Constraints I

Constraint (13.6) — Each ship arrives at its destination:

$$\sum_{i \in N_k} x_{id(k)k} = 1 \quad \forall k \in K \quad (13.6)$$

Every ship must terminate its voyage at the artificial destination node. Combined with (13.4) and (13.5), this ensures a valid route from $o(k)$ to $d(k)$ for each ship.

Constraint (13.7) — Load increases at a pickup node j :

$$l_{ik} + Q_j - l_{jk} - Q_k^C(1 - x_{ijk}) \leq 0 \quad \forall k \in K, (i, j) \in A_k, j \in N_k^P \quad (13.7)$$

Here l_{ik} is the load on ship k *after departing* node i . At a pickup node j , the cargo Q_j is added: $l_{jk} \geq l_{ik} + Q_j$. The term $Q_k^C(1 - x_{ijk})$ is a **“big-M” relaxation**: if ship k does *not* use arc (i, j) (i.e., $x_{ijk} = 0$), this term adds a large positive number, making the constraint trivially satisfied. The constraint only “activates” on arcs actually used.

The MPDP Mathematical Model: Load and Time Constraints II

Constraint (13.8) — Load decreases at a delivery node:

$$l_{ik} - Q_j - l_{n+j,k} - Q_k^C(1 - x_{i,n+j,k}) \leq 0 \quad j \in N_k^P \quad (13.8)$$

At a delivery node (index $n + j$, where n is the number of cargoes and j is the index of the associated pickup), the load decreases by Q_j . The pair (pickup node j , delivery node $n + j$) together represent one complete cargo movement.

Constraints (13.9)–(13.10) — Capacity bounds:

$$\sum_{j \in N_k} Q_j x_{ijk} \leq l_{ik} \leq \sum_{j \in N_k} Q_k^C x_{ijk} \quad \forall k \in K, i \in N_k^P \quad (13.9)$$

$$0 \leq l_{n+i,k} \leq \sum_{j \in N_k} (Q_k^C - Q_i) x_{n+i,jk} \quad \forall k \in K, i \in N_k^P \quad (13.10)$$

These ensure the load is between the cargo quantity (at least as much as has been picked up) and the ship's capacity Q_k^C at all times.

The MPDP Mathematical Model: Load and Time Constraints III

Constraint (13.11) — Time-window feasibility:

$$t_{ik} + T_{ik}^P + T_{ijk}^S - t_{jk} - M_{ijk}(1 - x_{ijk}) \leq 0 \quad \forall k \in K, (i, j) \in A_k \quad (13.11)$$

Here T_{ik}^P (T sub ik superscript P) is the **port handling time** (loading or unloading) at node i for ship k ; T_{ijk}^S is the **sailing time** from i to j at the ship's standard speed; and M_{ijk} is a big-M constant. The constraint says: if ship k travels from i to j , then $t_{jk} \geq t_{ik} + T_{ik}^P + T_{ijk}^S$ (it cannot arrive at j before it has finished at i and sailed across).

Constraint (13.12) — Pickup before delivery (pairing):

$$\sum_{j \in N_k} x_{ijk} - \sum_{j \in N_k} x_{n+i,jk} = 0 \quad \forall k \in K, i \in N_k^P \quad (13.12)$$

Ship k must visit the pickup node of cargo i if and only if it also visits the delivery node. You cannot deliver what you have not picked up, and you cannot pick up without delivering.

The MPDP Mathematical Model: Load and Time Constraints IV

Key Insight: The Big-M Technique

Constraints (13.7), (13.8), and (13.11) all use the “big-M” trick. A big constant M is added or subtracted based on a binary decision variable. This conditionally deactivates a constraint when a particular route arc is not used. The choice of M affects numerical stability: M should be as small as possible while still being large enough to make the constraint trivially feasible when the arc is unused.

The MPDP Mathematical Model: Load and Time Constraints V

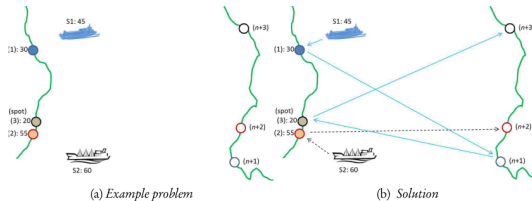


Figure 13.1. Example of multi-vehicle pickup-and-delivery problem and its solution.

Figure 13.1(b) shows a possible solution to the problem, where ship 1 first transports cargo 1 and then sails back (e.g., over the Atlantic Ocean) to service spot cargo 3, while ship 2 carries cargo 2. If we had included time windows, it is easy to imagine that ship 1 would have arrived at the pickup port of the optional spot cargo too late, so that one would have had to reject that cargo, thus missing the revenue from transporting it.

An excerpt from a real ship routing and scheduling working sheet used by a shipping company in the planning of ship routes and schedules is provided in Figure 13.2 (ship names and other information have been modified to preserve confidentiality). The pickup ports are divided into four different regions: United Kingdom and Continental Europe (UK/CONT), Mediterranean Sea (MEDIT), Caribbean Sea (CARIB), and United States and Gulf of Mexico (US/GULF). The cargoes are given in the upper row of the table, based on the locations of their pickup ports. Each cargo is specified by its pickup and delivery ports, its weight, its product type (i.e., MTBE or methanol), and the time window during which pickup can start. In the lower row, the available ships based in each region

The MPDP Mathematical Model: Load and Time Constraints VI

Figure: Example from the book: a tramp routing instance showing ship routes connecting pickup and delivery ports. (Reproduced from page 387 of the source text.)

Extension: Split Loads (Cargo Splitting) I

In the standard formulation above, each cargo is assigned to *exactly one* ship. However, some industrial problems require **cargo splitting**: distributing a single cargo across multiple ships.

Why Split Loads?

Two practical situations motivate cargo splitting:

- **Oversized cargo**: cargo quantity Q_i exceeds any single ship's capacity Q_k^C . Without splitting, the cargo cannot be served at all by the own fleet.
- **Delivery urgency**: splitting across two ships allows partial delivery sooner, satisfying time-window constraints that a single ship would miss.

Extension: Split Loads (Cargo Splitting) II

cargo_splitting.png

Extension: Variable Sailing Speed I

Real-world ship routing involves a powerful lever unavailable in road VRP: the ship's operator can **freely choose the speed** on each leg. This matters because the relationship between speed and fuel consumption is strongly non-linear, making slow steaming economically attractive.

The Cubic Fuel Consumption Law

Let v (v , for velocity measured in **knots**, where 1 knot is approximately 1.85 km/h) be the speed of a ship on a particular leg. The fuel consumption rate $F(v)$ (in tonnes of fuel per day) is approximately:

$$F(v) = \alpha \cdot v^3 \quad (1)$$

where α (alpha) is a ship-specific constant. This cubic relationship means: **reducing speed by 10%** reduces fuel consumption by approximately $1 - 0.9^3 \approx 27\%$.

Extension: Variable Sailing Speed II

How to Read the Cubic Law

If you double the speed ($v \rightarrow 2v$), fuel consumption increases *eightfold* ($2^3 = 8$). Conversely, if you halve the speed, fuel consumption falls to one-eighth. This strongly non-linear relationship makes slow steaming — operating at 12–14 knots instead of the design speed of 18–20 knots — financially very attractive when fuel prices are high. The trade-off is that slower ships spend more days at sea, which may cause them to miss time windows or reduce the number of voyages completed per year.

Extension: Variable Sailing Speed III

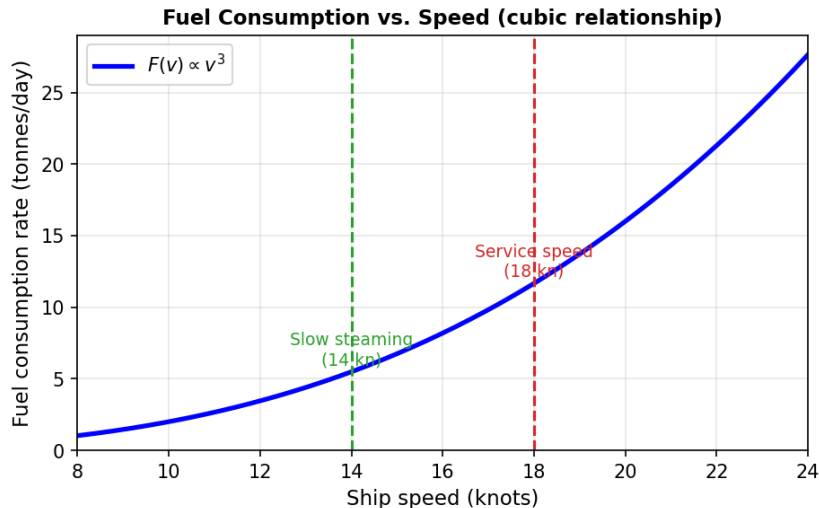


Figure: Left panel: total cost as a function of speed (fuel cost rises as v^3 while time-related cost falls as $1/v$; the optimum balances the two). Right panel: fuel consumption rate versus speed, with specific values annotated at ²⁷/₁

Solution Methods: Exact Approaches I

The MPDP is NP-hard (it generalises the TSP), so exact algorithms can only solve instances of moderate size. Two main exact approaches appear in the literature.

Branch-and-Bound (B&B)

The standard MIP formulation (13.1)–(13.15) can be solved directly by an off-the-shelf MIP solver (CPLEX, Gurobi) using **branch-and-bound**. The solver relaxes integrality constraints on x_{ijk} , solves the LP relaxation, then **branches** on a fractional variable (fixing it to 0 or 1 in two sub-problems) and recursively solves each. Valid inequalities (cuts) tighten the LP bound.

For ship routing, direct B&B solves instances with up to about 30 cargoes and 5–6 ships within one hour on modern hardware.

Solution Methods: Exact Approaches II

Branch-and-Price (B&P) with Column Generation

A more powerful exact method is **branch-and-price**, which combines branch-and-bound with **column generation**.

The key insight is that instead of working with individual arc variables x_{ijk} , the master problem works with *complete ship routes* (sequences of cargo pickups and deliveries) as columns. The number of possible columns is exponentially large, but column generation finds only the “useful” ones.

Algorithm: Column Generation for Ship Routing

- 1 Start with a small pool of feasible routes (columns) for each ship.
- 2 **Solve the Restricted Master Problem (RMP)**: select the best subset of routes from the current pool. The LP relaxation of the RMP yields **dual prices** π_i (π_i sub i) for each cargo coverage constraint. These prices measure how much the objective would improve by covering cargo i more cheaply.
- 3 **Solve the Pricing Subproblem** (one per ship): find the route with the most negative *reduced cost*:

$$\bar{c}_r = c_r - \sum_i \pi_i a_{ir}$$

where c_r is the voyage cost of route r and a_{ir} (a_{ir} sub ir) equals 1 if route r serves cargo i . The

Solution Methods: Heuristic Approaches I

For large real-world instances (more than 50 cargoes, more than 10 ships), exact methods are too slow. The chapter describes several powerful heuristics.

Solution Methods: Heuristic Approaches II

Tabu Search (TS)

Tabu search is a **local search metaheuristic** that systematically explores the neighbourhood of the current solution, accepting improving moves but also temporarily allowing “uphill” moves to escape local optima.

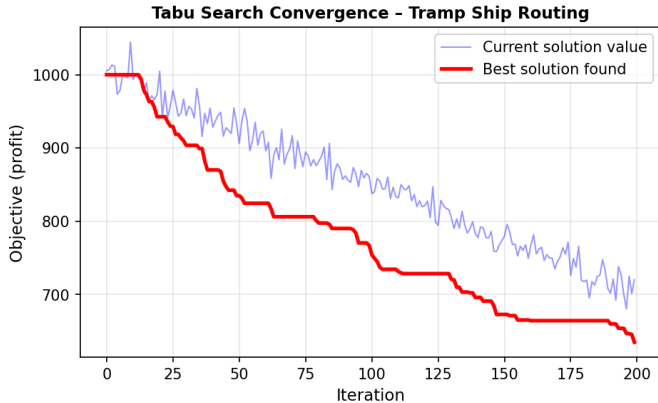
The name “tabu” comes from the Polynesian word for “forbidden.” A **tabu list** records recently visited moves. A move is forbidden if it reverses a recent change, preventing the algorithm from cycling.

In ship routing, a **neighbourhood move** is one of:

- **Reinsertion**: remove cargo i from ship k 's route and insert it into ship k' 's route (or into the spot market).
- **Swap**: exchange cargo i on ship k with cargo j on ship k' .
- **2-opt within a ship**: reverse a sub-sequence of port visits on the same ship's route.

Christiansen, Fagerholt, and Ronen (2004) apply TS to a real-world tramp problem and achieve solutions within 2–5% of the optimum for instances that exact methods cannot solve.

Solution Methods: Heuristic Approaches III



Solution Methods: Heuristic Approaches IV

Figure: Typical tabu search convergence. The blue line is the current solution value (can move uphill, i.e., worsen temporarily); the red line tracks the best solution found so far. The algorithm escapes several local optima before finding the final best.

Solution Methods: Heuristic Approaches V

Adaptive Large Neighbourhood Search (ALNS)

ALNS extends tabu search by using multiple **destroy-and-repair** operators, selecting the most effective one adaptively based on recent performance history.

The **destroy phase** removes a subset of cargoes from their current ship assignments using operators such as: random removal, worst-cargo removal (remove the cargo contributing least profit), or route-based removal (remove all cargoes on a selected ship).

The **repair phase** reinserts removed cargoes using a greedy or regret-based heuristic. The **regret- k** heuristic inserts the cargo for which the difference between the best and second-best insertion position is largest (high regret = urgency to insert now).

Operators are weighted by recent success; the weight of operator o is updated as:

$w_o \leftarrow (1 - \rho) w_o + \rho \sigma_o$, where ρ (rho) is a learning rate and σ_o is the score earned (high score if the operator found a new best solution).

ALNS has been shown to outperform single-neighbourhood TS on the MPDP, especially for instances mixing mandatory and optional cargoes.

Solution Methods: Heuristic Approaches VI

benchmark_comparison.png

Section 13.3: Maritime Inventory Routing (MIR) I

Maritime Inventory Routing (MIR) integrates **inventory management at ports** with **ship scheduling**. Instead of treating cargoes as discrete units (as in MPDP), MIR models the supply chain as a continuous flow: production ports generate stock; consumption ports draw down stock; ships shuttle material between them. The operator must ensure no port *runs out of stock* (“stock-out”) or *overflows* its storage tanks — while minimising total shipping cost.

Where MIR Arises in Practice

MIR is the operational planning model for:

- Oil companies managing fleets of product tankers between refineries and regional fuel-distribution terminals.
- LNG (liquefied natural gas) producers shipping from export terminals to regasification plants worldwide.
- Chemical companies transporting feedstocks and intermediates between offshore and onshore plants.

In all cases, the quantity loaded at each port visit is a *decision variable*, and the timing of visits must be coordinated with the inventory dynamics at each port.

Section 13.3: Maritime Inventory Routing (MIR) II

`mirp_supply_chain.png`

13.3.1 MIR Problem Description and Example I

Ports and inventory dynamics. Let N denote the set of ports. Each port $i \in N$ is either a **production port** ($l_i = +1$: stock accumulates between ship visits, as oil wells continue pumping into tanks) or a **consumption port** ($l_i = -1$: stock falls between visits, as the plant draws material from tanks).

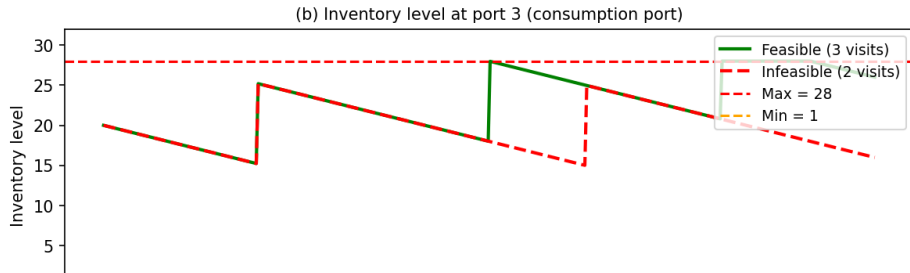
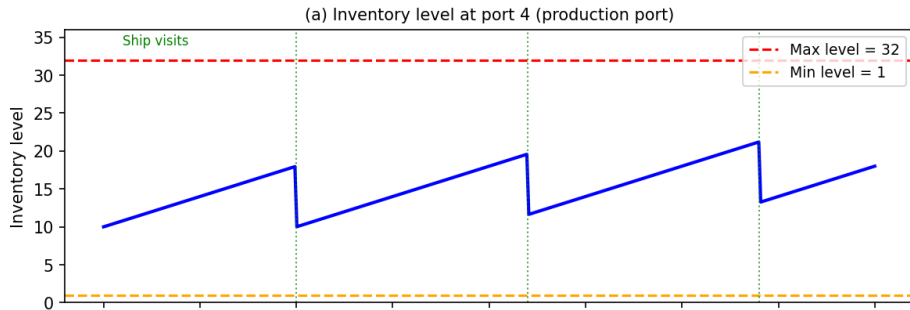
The inventory s_{im} (s sub i - m) at port i at the time of visit m satisfies a simple balance equation. Between consecutive visits at times $t_{i,m-1}$ and t_{im} , the inventory changes at rate R_i (production or consumption rate, in tonnes per day). When a ship visits, a quantity q_{imk} is loaded or unloaded.

Inventory Level Intuition

At a **production port**, inventory *rises* steadily between ship visits (oil is pumped from wells into storage tanks), then *drops sharply* when a ship arrives and loads. If visits are too infrequent, the tanks overflow. If visits are too frequent, ships sail partially empty.

At a **consumption port**, inventory *falls* steadily between visits (material is consumed in the plant), then *rises* when a ship delivers more. If visits are too infrequent, the port runs out of material (stock-out) and production halts — a very costly event.

13.3.1 MIR Problem Description and Example II



13.3.2 MIR Mathematical Model I

The full MIR model extends the MPDP formulation with additional variables and constraints that capture inventory dynamics. The index m tracks the *sequence* of visits to each port during the planning horizon.

Additional sets and variables:

- M_i — the ordered set of visit slots to port i ($m = 1, 2, \dots, |M_i|$).
- M_{ik} — the subset of visits to port i that ship k could perform.
- $x_{imjnk} \in \{0, 1\}$ — equals 1 if ship k travels from port-visit (i, m) directly to port-visit (j, n) . This is the arc-flow variable in the *time-space* formulation.
- $w_{im} \in \{0, 1\}$ — equals 1 if visit slot m at port i is **skipped** (no ship visits during that slot).
- $q_{imk} \geq 0$ — quantity loaded (production port) or unloaded (consumption port) by ship k at port i visit m .
- $s_{im} \geq 0$ — inventory level at port i at visit time m .
- t_{im} — time (days from start) of the m -th visit to port i .

13.3.2 MIR Mathematical Model II

Coverage (13.41)

$$\sum_{k \in K} \sum_{j \in N_k} \sum_{n \in M_{jk}} x_{imjnk} + w_{im} = 1 \quad \forall i \in N, m \in M_i \quad (13.41)$$

Every visit slot is either served by exactly one ship or is skipped. The left sum counts how many ships visit port i at slot m ; adding the skip indicator w_{im} brings the total to exactly 1.

13.3.2 MIR Mathematical Model III

Inventory Balance (13.52–13.53)

$$s_{i1} - R_i t_{i1} = S_i^0 \quad \forall i \in N \quad (13.52)$$

$$s_{i,m-1} - \sum_{k \in K} l_i q_{i,m-1,k} + R_i(t_{im} - t_{i,m-1}) - s_{im} = 0 \quad m \geq 2 \quad (13.53)$$

Constraint (13.52) initialises the inventory: at the first visit (time t_{i1}), the inventory s_{i1} equals the initial level S_i^0 plus the production/consumption since time 0. Constraint (13.53) is the **inventory balance**: the inventory at visit m equals the inventory at visit $m-1$, adjusted for any loading/unloading ($l_i q_{i,m-1,k}$, where $l_i = +1$ if production port and -1 if consumption port), plus the natural accumulation/depletion over the inter-visit time ($t_{im} - t_{i,m-1}$) at rate R_i .

13.3.2 MIR Mathematical Model IV

How to Read the Inventory Balance

Think of the inventory as a bank account. At a production port: money (inventory) accumulates at a steady rate R_i (interest), and ships “withdraw” (load) q_{imk} at each visit. The balance equation simply says: ending balance = starting balance + accrual - withdrawal. At a consumption port, the rate R_i is a debit (withdrawal); ship visits are credits (deliveries). The constraint enforces that this bank statement is internally consistent.

13.3.2 MIR Mathematical Model V

Inventory Bounds (13.54–13.55)

$$\underline{S}_i \leq s_{im} \leq \bar{S}_i \quad \forall i \in N, m \in M_i \quad (13.54)$$

$$\underline{S}_i \leq s_{im} - \sum_{k \in K} l_i q_{im|M_i|k} + R_i(T - t_{im}) \leq \bar{S}_i \quad m = |M_i| \quad (13.55)$$

Constraint (13.54) enforces $[\underline{S}_i, \bar{S}_i]$ at each visit. Constraint (13.55) additionally checks the *end-of-horizon* inventory — after the last visit at time $t_{i|M_i|}$, inventory must still be within bounds at the horizon end T . This prevents the plan from scheduling a large delivery just before the horizon and then leaving the port in an infeasible state afterwards.

13.3.2 MIR Mathematical Model VI

Load Feasibility (13.46–13.49)

Analogous to the MPDP constraints (13.7)–(13.9), the MIR model includes constraints ensuring the load l_{imk} on ship k (after visiting port i at slot m) stays within capacity $[0, Q_k^C]$:

$$l_{imk} + l_j q_{jnk} - l_{jnk} - Q_k^C (1 - x_{imjnk}) \leq 0 \quad (13.46)$$

$$l_{imk} + l_j q_{jnk} - l_{jnk} + Q_k^C (1 - x_{imjnk}) \geq 0 \quad (13.47)$$

$$q_{imk} \leq l_{imk} \leq \sum_{j,n} Q_k^C x_{imjnk} \quad (13.48)$$

These constraints “link” the load on the ship to the quantity loaded or unloaded at each port visit, enforcing consistency between the ship’s physical state and the inventory at each port.

MIR Solution Methods I

The MIR model (13.40)–(13.59) is a large MIP. For a problem with 4 ports, 5 ships, and a 40-day horizon, decision variables can exceed 100,000. Several solution approaches have been developed.

Arc-Flow vs. Path-Flow Formulations

The model described above is an **arc-flow** formulation: binary variables x_{imjnk} for individual arcs between port-visit slots. An alternative is the **path-flow** formulation, which uses variables $\lambda_r \geq 0$ representing the usage of complete ship *schedules* (full sequences of port visits for one ship over the entire horizon). Path-flow formulations have tighter LP relaxations and suit column generation well, but the pricing subproblem (finding the best ship schedule) is itself a complex shortest-path problem with inventory side constraints.

Heuristics for MIR

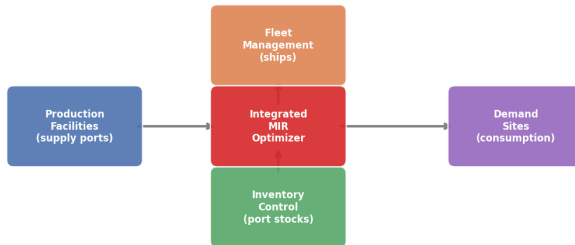
- **Decomposition:** first determine how many times each port needs to be visited (a single-port inventory problem), then solve a smaller routing problem.
- **Rolling horizon:** solve MIR over a short window (e.g., 2 weeks), fix the first week's decisions, and roll forward. This handles instances that are too large for a single solve.
- **Column generation:** generate ship schedules (columns) that cover feasible sequences of port visits, with inventory implications computed implicitly within the pricing subproblem.
- **Approximate Dynamic Programming (ADP):** for long-horizon MIR, Papageorgiou et al. (2014/2015) develop an ADP approach that approximates the value function using basis functions, enabling very long horizons (months to years) with multiple ships.

MIR Solution Methods III

Key Insight: MIR Captures Full Supply-Chain Cost

Traditional ship routing models minimise shipping cost alone. MIR additionally accounts for: the cost of *holding inventory* (capital tied up in stock), the cost of *stock-outs* (lost production when material runs out), and the cost of *storage overflow* (spillage or forced venting of gases). Studies show MIR solutions can reduce total supply-chain cost by 5–15% compared with separately optimising routing and inventory.

Maritime Inventory Routing: Integrated Supply Chain



MIR Solution Methods IV

Figure: The MIR optimizer integrates three traditionally separate decisions: production scheduling at supply ports (blue), fleet management (orange), and inventory control at demand ports (green). The integrated approach avoids the suboptimality that arises when each layer optimises independently.

MIR Extensions: Multiple Products and Varying Quantities I

The MIR setting opens several practically important extensions.

Multiple Products

When ships carry multiple products in separate compartments (as in a chemical tanker with several segregated tanks), the model must track inventory separately for each product. Siswanto, Essam, and Sarker (2011) develop a MIP model and metaheuristic for this **multi-compartment MIR**. Key additional constraints specify: which product goes in which compartment, cleaning time between products, and stowage restrictions.

Varying Production and Consumption Rates

In reality, production rates at oil fields vary with geological depletion, and consumption rates at power plants vary with demand. A robust MIR model treats rates $R_i(t)$ as time-varying parameters. This turns the inventory balance into a time-dependent difference equation and typically requires discretising the planning horizon into time periods.

MIR Extensions: Multiple Products and Varying Quantities II

Integration with Production Scheduling

At the most integrated level, the MIR model is embedded within a **production-routing** model where the production schedule at each plant is also a decision variable. For example, a refinery might choose *when* to produce which products based on ship schedules. This integration further reduces total cost but also greatly increases model complexity. Ronen (2002) and Dauzere-Peres et al. study this integration in the context of petroleum distribution.

Takeaway: MIR is the Richest VRP Variant in This Chapter

MIR simultaneously decides: which ships serve which ports, in what order, at what times, loading how much. All these decisions interact through the inventory balance equation. The result is a combinatorially explosive problem that requires dedicated formulations and algorithms to solve at industry scale.

Section 13.4: Dynamic and Stochastic Ship Routing I

All models described so far assume that all information is **known in advance**. In reality, ship routing is subject to many sources of uncertainty:

- **Weather:** storms can delay a ship by several days on ocean routes; fuel consumption increases sharply in heavy seas.
- **Port congestion and closure:** berths can be unexpectedly unavailable due to maintenance, strikes, or tidal restrictions.
- **Spot market dynamics:** new cargo offers arrive continuously in real time, requiring immediate re-routing decisions.
- **Demand uncertainty:** in MIR settings, consumption rates at ports fluctuate with seasons, economic activity, or equipment failures.

Section 13.4: Dynamic and Stochastic Ship Routing II

Two Paradigms for Uncertainty

- ① **Dynamic (online) routing:** the plan is re-optimised each time new information arrives (event-driven re-planning, also called a “rolling horizon” or “model predictive control” approach).
- ② **Stochastic programming:** uncertainty is modelled *explicitly* via probability distributions and scenario trees. The optimiser finds first-stage decisions that perform well across all realisations of the uncertainty.

Section 13.4: Dynamic and Stochastic Ship Routing III

`dynamic_stochastic_timeline.png`

13.4.1 Dynamic Routing: Rolling Horizon Re-planning I

The most widely used practical approach to dynamic routing is the **rolling horizon** framework.

Rolling Horizon Algorithm

- ➊ At decision time t , gather all currently known information: positions of ships, confirmed cargoes, weather forecasts, port status, and pending spot cargo offers.
- ➋ Solve the static MPDP or MIR over a planning horizon $[t, t + H]$ (horizon length H , typically 2–4 weeks).
- ➌ **Execute only the first δ time units** of the resulting plan. These decisions are *committed* (binding).
- ➍ At time $t + \delta$, receive new information (new cargo offers, updated weather, port closures), and re-solve.
- ➎ Repeat until the end of the planning period.

The key design choices are:

- **Horizon length H :** too short leads to myopic decisions that miss profitable future cargo; too long is expensive to re-solve and the far future is very uncertain.

13.4.1 Dynamic Routing: Rolling Horizon Re-planning II

- **Commitment horizon** δ : how far ahead the plan is committed before the next re-optimisation. Smaller δ means more frequent re-optimisation and better responsiveness, but higher computational cost.
- **End-of-horizon value function**: some approaches add a term to the objective representing the *expected value of the fleet's position* at the end of the horizon, preventing artificially good decisions that leave ships in bad positions.

13.4.1 Dynamic Routing: Rolling Horizon Re-planning III

Worked Example: Rolling Horizon in Action

We manage 3 ships, horizon $H = 30$ days, commitment $\delta = 7$ days.

Day 0: solve MPDP for days 0–30. The plan assigns: Ship 1 to Cargo A (depart day 2, arrive day 9), Ship 2 to Cargo B (depart day 1, arrive day 8), Ship 3 on standby for optional spot cargoes.

Day 7: during the first 7 days, two events occurred: (a) a new spot cargo C appeared (worth \$200k; pickup day 10, delivery day 20); (b) Cargo B's destination port is closed until day 12. We re-solve the MPDP for days 7–37 incorporating these updates. New plan: Ship 2 rerouted to an alternative port; Ship 3 assigned to Cargo C's pickup.

The first 7 days of the new plan are now binding.

Key insight: rolling horizon handles disruptions naturally — the planner simply re-solves with updated data at each period. The quality of the solution degrades gracefully as disruptions become more frequent or severe.

Tirado et al. (2013) apply a rolling horizon heuristic based on ALNS to a dynamic industrial shipping problem with uncertain spot-cargo arrivals. Their method achieves solutions within 3–8% of a perfect-information (clairvoyant) benchmark even under high cargo-uncertainty levels.

13.4.2 Stochastic Programming for Ship Routing I

A more principled treatment of uncertainty uses **stochastic programming**, which models the future explicitly as a probability distribution over *scenarios*.

13.4.2 Stochastic Programming for Ship Routing II

Two-Stage Stochastic Programme

Let Ω (Omega, the set of all possible future scenarios) and p_ω (p sub ω (omega), the probability of scenario $\omega \in \Omega$). The two-stage stochastic formulation is:

$$\max_{x \in X} c^\top x + \sum_{\omega \in \Omega} p_\omega Q(x, \omega) \quad (3)$$

where:

- x is the **first-stage** (“here-and-now”) decision made *before* uncertainty is revealed (e.g., which mandatory cargoes to commit to, how to pre-position the fleet).
- X is the feasible set of first-stage decisions.
- $c^\top x$ is the first-stage revenue/cost.
- $Q(x, \omega)$ is the **second-stage** optimal value given first-stage decisions x and realised scenario ω (e.g., the optimal routing adjustment after seeing actual weather).
- $\sum_\omega p_\omega Q(x, \omega)$ is the **expected recourse value**: the average second-stage payoff across all scenarios.

13.4.2 Applications: Weather Routing and Port Disruptions I

Weather Routing

Ocean weather strongly affects transit time and fuel consumption. A ship in a force-8 storm may be delayed by 1–3 days and consume 30–50% more fuel than in calm conditions.

Weather routing models typically:

- 1 Obtain a probabilistic weather forecast (wind field, wave height, ocean current) over the planning horizon.
- 2 Model ship speed and fuel consumption as functions of wave height and wave angle (using *polar diagrams* from the ship's technical specification).
- 3 Optimise the route (sequence of waypoints) to balance transit time, fuel cost, and risk of delay.

Lo and Liu (2007) model weather routing as a stochastic dynamic programme on a discretised ocean grid. The optimal route may intentionally *deviate* from the great-circle (shortest-distance) path to avoid a storm, saving more in fuel and delay than the extra distance costs.

13.4.2 Applications: Weather Routing and Port Disruptions II

Stochastic Port Availability

Port congestion and berth availability are inherently stochastic. Modelling port availability as a random variable $\xi_{ik} \in \{0, 1\}$ (ξ , pronounced “ksee”, sub i - k , where 1 means the berth is available when ship k arrives at port i) leads to a **chance-constrained programme**:

$$\Pr(t_{ik} \in [\underline{T}_{ik}, \overline{T}_{ik}] \text{ and berth is available}) \geq 1 - \varepsilon \quad (5)$$

where ε (epsilon) is the *acceptable risk* of a time-window violation (e.g., $\varepsilon = 0.05$ for a 5% risk). This enforces *probabilistic* time-window compliance rather than hard deterministic limits, allowing the model to trade off schedule tightness against robustness to delays.

13.4.2 Applications: Weather Routing and Port Disruptions III

Key Insight: When to Use Stochastic vs. Dynamic Models

Use **stochastic programming** when: uncertainty can be quantified via probability distributions, the planning cycle is long enough to justify the extra computation, and first-stage decisions must be robust (e.g., long-term contracts). Use **dynamic rolling-horizon re-planning** when: uncertainty is revealed sequentially in real time, re-computation can be done quickly (minutes to hours), and there is flexibility to change the plan as new information arrives. In practice, both approaches are often combined: a stochastic model is used for strategic decisions and a rolling horizon for daily tactical adjustments.

Section 13.5: Conclusions and Future Research Directions I

Summary of the Chapter

This chapter has surveyed three main classes of ship routing and scheduling problems:

- 1 **Cargo Routing and Scheduling (MPDP):** assigning cargoes to ships and optimising routes and schedules. The core MIP model (13.1)–(13.15) captures mandatory vs. optional cargoes, fleet heterogeneity, capacity, and time windows. Exact methods (branch-and-price with column generation) solve medium instances; metaheuristics (ALNS, tabu search) handle large real fleets.
- 2 **Maritime Inventory Routing (MIR):** simultaneously optimising ship routes and port inventory levels. The MIR model (13.40)–(13.59) adds inventory balance equations to the MPDP framework. The integrated approach reduces total supply-chain cost by 5–15% compared with decoupled routing and inventory optimisation.
- 3 **Dynamic and Stochastic Routing:** handling weather, port congestion, and real-time cargo offers via rolling-horizon re-optimisation and two-stage stochastic programming.

Section 13.5: Conclusions and Future Research Directions II

What Makes Maritime Routing Distinctive

Compared with classical road VRP, maritime ship routing presents unique challenges:

- **Variable speed:** the cubic fuel–speed law creates a continuous optimisation dimension absent in road VRP.
- **Enormous cargo quantities:** a single voyage decision can move 100,000+ tonnes and generate millions in revenue, making every percentage-point of optimality improvement valuable.
- **Long planning horizons:** weeks to months require time-expanded graph models and rolling-horizon methods.
- **Supply-chain integration via MIR:** no direct road-VRP analogue combines production, shipping, and inventory control.
- **Environmental impact:** ships produce approximately 2–3% of global CO₂; route and speed optimisation directly reduces the carbon footprint of world trade.

Section 13.5: Conclusions and Future Research Directions III

Future Research Directions Identified in the Chapter

- ➊ **Larger MIR models:** current exact methods handle 4–6 ports; real supply chains involve 20–50 ports. Better decompositions and tighter formulations are needed.
- ➋ **Dynamic MIR:** combining real-time re-planning with inventory tracking remains largely open.
- ➌ **Integrated speed optimisation:** jointly optimising routes, cargo assignment, and speed profiles is computationally challenging and practically impactful.
- ➍ **Environmental objectives:** incorporating CO₂ caps, sulphur emission control areas (ECAs), and port emissions regulations as hard constraints or additional objectives.
- ➎ **Autonomous shipping:** as autonomous ships become viable, real-time algorithms that solve routing problems at sea with incomplete information will be essential.

Section 13.5: Conclusions and Future Research Directions IV

Key References for Further Study

- Christiansen, Fagerholt, Nygreen, Ronen (2007): comprehensive survey of maritime transportation OR (Handbooks in OR).
- Fagerholt, Gausel, Rakke, Ronen (2010/2015): variable-speed optimisation for ship routing.
- Papageorgiou, Cheon, Nemhauser (2014/2015): approximate dynamic programming for long-horizon MIR.
- Tirado, Hvattum, Fagerholt, Cordeau (2013): heuristics for dynamic and stochastic industrial shipping.
- Stalhåne, Andersson, Christiansen, Cordeau, Desaulniers (2012): branch-and-cut for routing with split loads.

Python: Greedy Tramp Routing Heuristic (Part 1)

Listing 1: Greedy construction heuristic for the tramp pickup-and-delivery problem. Implements the insertion-based procedure from Section 13.2.

```
import math

def haversine_nm(lat1, lon1, lat2, lon2):
    """Great-circle distance in nautical miles between two (lat, lon) points."""
    R = 3440.065 # Earth radius in nm
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlam = math.radians(lon2 - lon1)
    a = (math.sin(dphi/2)**2
          + math.cos(phi1) * math.cos(phi2) * math.sin(dlam/2)**2)
    return 2 * R * math.asin(math.sqrt(a))

def fuel_cost_usd(distance_nm, speed_kn=14.0,
                  alpha=0.002, fuel_price_per_tonne=600):
    """
    Total fuel cost (USD) for one leg.
     $F(v) = \alpha * v^3$  (tonnes/day); voyage time = distance_nm / (speed_kn*24) days.
    """
    days = distance_nm / (speed_kn * 24) # sailing time in days
    tonnes = alpha * (speed_kn ** 3) * days # total fuel consumed
    return tonnes * fuel_price_per_tonne

class Cargo:
    def __init__(self, cid, pickup, delivery, qty, rev_per_tonne,
```

Python: Greedy Tramp Routing Heuristic (Part 2)

Listing 2: Continued: greedy insertion loop and example run.

```
def greedy_insertion(ships, cargoes):  
    """  
    Assign cargoes to ships greedily: at each step, choose the (cargo, ship)  
    pair that maximises marginal profit (revenue - extra voyage cost).  
    Mandatory cargoes are prioritised; unservable ones go to the spot market.  
    """  
  
    assignments = {s.kid: [] for s in ships}  
    spot_market = []  
  
    # Sort: mandatory cargoes first; within mandatory, highest revenue first  
    sorted_cargoes = sorted(  
        cargoes,  
        key=lambda c: (-int(c.mandatory), -c.rev * c.qty)  
    )  
  
    for cargo in sorted_cargoes:  
        best_gain, best_ship = -math.inf, None  
  
        for ship in ships:  
            current_load = sum(c.qty for c in assignments[ship.kid])  
            if current_load + cargo.qty > ship.capacity:  
                continue # would exceed capacity -- skip  
  
            # Marginal cost = sailing to pickup + pickup to delivery  
            if assignments[ship.kid]:
```

Python: Maritime Inventory Routing Simulation

Listing 3: Simulation of port inventory levels under a given MIR schedule. Implements the inventory balance (constraints 13.52–13.53).

```
def simulate_inventory(port_name, rate, s0, s_min, s_max,
                      visit_times, quantities, T=40):
    """
    Simulate inventory at one port over [0, T] days.
    rate          : production (+) or consumption (-) rate in units/day
    s0             : initial inventory at t=0
    s_min, s_max  : inventory bounds
    visit_times    : list of days when a ship visits
    quantities     : quantity loaded (positive) or unloaded (negative) at each visit
    """
    events = sorted(zip(visit_times, quantities))
    t_current = 0.0
    s_current = s0
    trajectory = [(0.0, s0)]    # (time, inventory) pairs
    violations = []

    for (t_visit, q) in events:
        # Evolve inventory from last event to just before ship arrival
        s_before = s_current + rate * (t_visit - t_current)
        trajectory.append((t_visit, s_before))
        if not (s_min <= s_before <= s_max):
            violations.append(('before visit', t_visit, s_before))

    # Ship loads / unloads
```

Python: Speed Optimisation for One Leg

Listing 4: Finding the cost-optimal speed on a single leg, balancing cubic fuel cost against time-charter (daily) cost.

```
import numpy as np
from scipy.optimize import minimize_scalar

def total_leg_cost(v, distance_nm, alpha=0.002,
                  fuel_price=600, time_charter_rate=20000):
    """
    Total cost for one sailing leg at constant speed v (knots).
    Components:
    1. Fuel cost:  $\alpha * v^3 * (\text{transit\_days}) * \text{fuel\_price}$ 
    2. Time-charter (opportunity) cost:  $\text{time\_charter\_rate} * \text{transit\_days}$ 
    """
    transit_days = distance_nm / (v * 24) # 24 hours per day
    fuel_tonnes = alpha * (v ** 3) * transit_days
    fuel_cost = fuel_tonnes * fuel_price
    charter_cost = time_charter_rate * transit_days # USD/day * days
    return fuel_cost + charter_cost

def optimal_speed(distance_nm, v_min=8.0, v_max=25.0,
                  alpha=0.002, fuel_price=600, tc_rate=20000):
    """Find the cost-minimising speed on a given leg."""
    result = minimize_scalar(
        total_leg_cost,
        bounds=(v_min, v_max),
        method='bounded',
```

Chapter Summary: Key Concepts and Takeaways I

This chapter has presented ship routing and scheduling as a rich family of combinatorial optimisation problems with significant real-world impact.

Core Message 1: Three Shipping Modes, Three Problem Classes

Industrial, liner, and tramp shipping each give rise to qualitatively different optimisation problems. Tramp and industrial shipping lead to MIP-based pickup-and-delivery models; liner shipping leads to network design problems. Maritime Inventory Routing bridges shipping and production planning into a unified optimisation model.

Core Message 2: The MIP Model (13.1)–(13.15) is the Foundation

Understanding the base MPDP formulation deeply makes all extensions accessible. The objective (13.1) balances revenue from cargoes against voyage costs and spot-market hiring costs. Constraints (13.2)–(13.6) define valid routes; (13.7)–(13.10) enforce capacity; (13.11)–(13.12) enforce time windows and pickup-before-delivery.

Chapter Summary: Key Concepts and Takeaways II

Core Message 3: Column Generation is the Workhorse Solver

For medium-to-large instances, branch-and-price (combining B&B with column generation) is the standard exact method. It avoids exponential route enumeration by generating only “useful” ship routes at each step. For very large or dynamic instances, ALNS and tabu search provide high-quality solutions efficiently.

Core Message 4: Uncertainty Requires Explicit Treatment

Weather, port disruptions, and real-time cargo offers make ship routing inherently dynamic. Rolling-horizon re-optimisation is the most practical approach; stochastic programming provides theoretically stronger guarantees at higher computational cost. The Value of the Stochastic Solution ($VSS \approx 5\text{--}15\%$) quantifies the benefit of modelling uncertainty explicitly.

Chapter Summary: Key Concepts and Takeaways III

Connection to Other VRP Chapters

Ship routing inherits the pickup-and-delivery structure from the classical PDVRP literature, the column-generation methodology from set-partitioning models, the metaheuristic framework from ALNS/TS research, and the stochastic modelling from two-stage stochastic programming. The maritime setting provides an especially rich application where all these techniques combine — and where the economic and environmental stakes are highest.

Chapter Summary: Key Concepts and Takeaways IV

benchmark_comparison.png

Selected Bibliography I



M. Christiansen, K. Fagerholt, and D. Ronen, *Ship routing and scheduling: Status and perspectives*, Transportation Science, 38 (2004), pp. 1–18.



M. Christiansen, K. Fagerholt, B. Nygreen, and D. Ronen, *Maritime transportation*, in C. Barnhart and G. Laporte (eds.), *Handbooks in Operations Research and Management Science*, Vol. 14, 2007, pp. 189–284.



K. Fagerholt, G. Gausel, J. G. Rakke, and H. N. Ronen, *Maritime routing and speed optimisation with emission control areas*, Transportation Research Part C, 52 (2015), pp. 57–73.



D. J. Papageorgiou, G.-H. Cheon, and G. L. Nemhauser, *Approximate dynamic programming for a class of long-horizon maritime inventory routing problems*, Transportation Science, 49 (2015), pp. 870–883.



G. Tirado, L. M. Hvattum, K. Fagerholt, and J.-F. Cordeau, *Heuristics for dynamic and stochastic routing in industrial shipping*, Computers & Operations Research, 40 (2013), pp. 253–263.

Selected Bibliography II

-  M. Stålhane, H. Andersson, M. Christiansen, J.-F. Cordeau, and G. Desaulniers, *A branch-and-cut method for a ship routing and scheduling problem with split loads*, Computers & Operations Research, 39 (2012), pp. 3361–3375.
-  H. K. Lo and C. W. Liu, *Adaptive ship routing through stochastic ocean currents: General formulations and empirical results*, Transportation Research Part B, 41 (2007), pp. 6–24.
-  J. Vidal, *CO₂ output from shipping twice as much as airlines*, 2007.
-  UNCTAD, *Review of maritime transport*, 2011, United Nations, New York and Geneva.
-  L. M. Hvattum and K. Fagerholt, *Rolling horizon and scenario-based heuristic solution methods for stochastic maritime inventory routing problems*, European Journal of Operational Research, 207 (2010), pp. 1026–1035.
-  H. Andersson, A. Hoff, M. Christiansen, G. Hasle, and A. Løkketangen, *Industrial aspects and literature survey: Combined inventory management and routing*, Computers & Operations Research, 37 (2010), pp. 1515–1536.